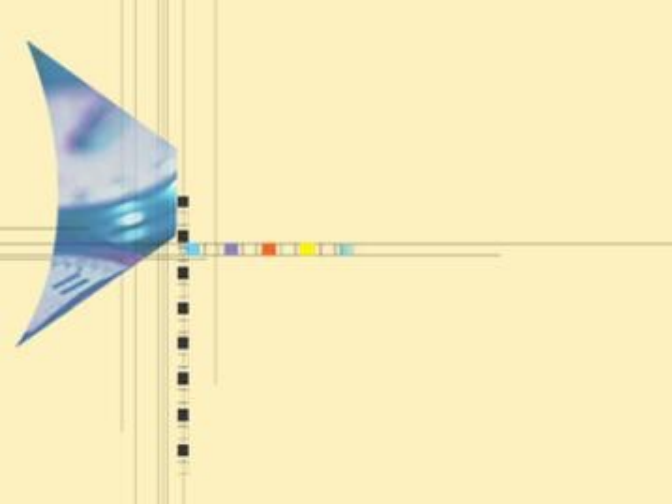




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

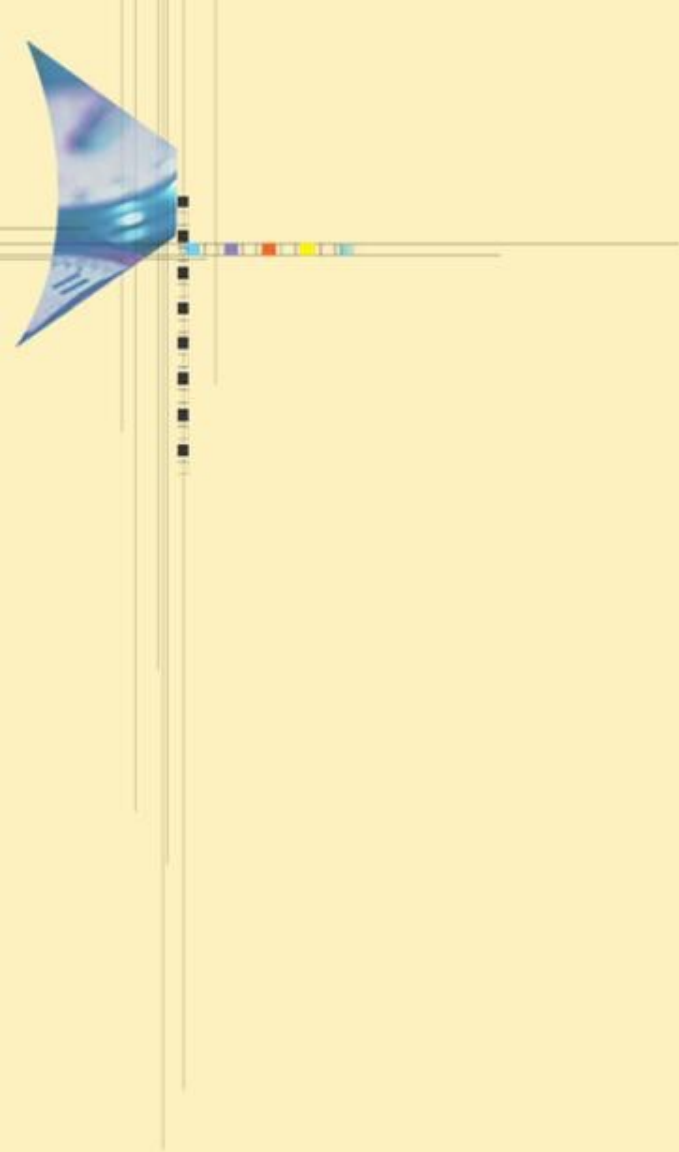
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/23

武汉大学计算机学院

武汉大学
Wuhan University





第九讲

分支限界法



主要内容

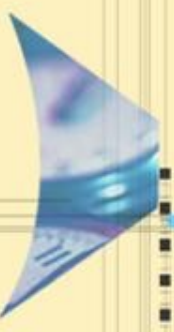
9.1 引言

9.2 主要检索策略

9.3 分支限界算法设计思想

9.4 分支限界法的典型应用





主要内容

9.1 引言

9.1.1 分支限界法简介

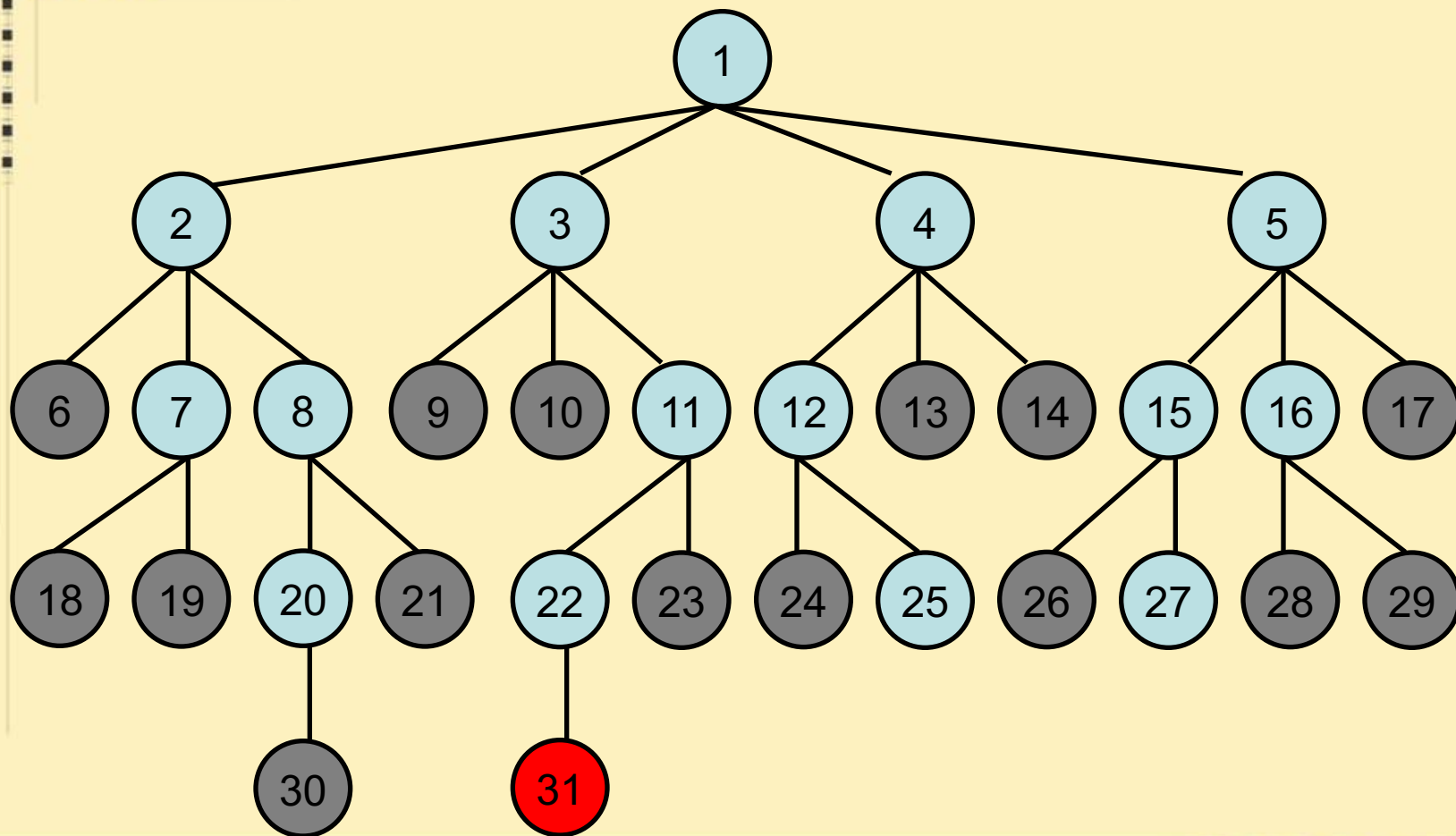
9.1.2 分支限界和回溯策略的异同点



9.1.1 分支限界法(Branch and Bound)简介

- 分支-限界法：生成当前E-结点（扩展结点）的全部子结点（分支），利用剪枝函数对不能生成含答案结点的子树剪枝（限界）的状态空间树检索方法。即，每当访问一个结点（E-结点）时，会生成这个结点的所有子结点（广度优先搜索）。
- 其核心：多个活节点的组织，E-节点的检索；剪枝函数的设计
- 主要检索策略：
 - FIFO(first in first out)检索：类似于BFS的状态空间检索，活结点采用普通队列组织
 - LIFO(last in first out)检索：类似于D-检索的状态空间检索，活结点表采用栈组织
 - 最优检索：最小耗费（最大收益）的方式检索，活结点表采用优先级队列组织

例子：4-皇后问题的FIFO分支限界法



9.1.2 分支界限法与回溯法异同点

相同点：

- 都要求问题的解能表示成有限元组的形式，元组分量取值范围有穷可枚举。解空间有穷。
- 都是通过系统地搜索解空间求问题解的搜索算法。为便于搜索解，都是通过将解空间组织成隐式的状态空间树的形式。
- 在搜索过程中，都通过约束条件对不可能导致合法解的部分进行剪枝，从而使求解时间大大缩短（与蛮力算法最显著的区别）

9.1.2 分支界限法与回溯法异同点

不同点:

- 回溯法一个活结点可以有 multiple 机会成为E结点。一次生成E-结点的1个子结点，新生成的子结点马上成为新的E-结点。以深度优先的方式搜索解空间；
- 分支限界法每一个活结点只有一次机会成为扩展结点。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。以广度优先或最小耗费（最大收益）的方式搜索解空间。
- 回溯法可用来寻找一个或者所有可行解，也可以用于找最优解（效率不是太高）；分支限界一般只找一个可行解，更多情况用于找最优解。



主要内容

9.1 引言

9.2 主要检索策略

9.3 分支限界算法设计思想

9.4 分支限界法的典型应用

9.2 检索策略

活检点的检索策略主要有：

- FIFO检索：队列
- LIFO检索：堆栈（回溯）
- 最优检索：优先级队列

FIFO/LIFO检索策略的缺陷

- 对下一个E-结点的选择相当死板，而且在某种意义上是盲目的。
- 这种选择对于有可能快速达到一个答案结点的结点没有任何优先权。

举例：15-谜问题

- 问题描述：在一个分成 4×4 的方形棋盘上放有15块编号为1, 2, ..., 15的牌，给定这些牌的初始排列（例图1），要求通过一系列合法的移动(只有相邻于空格的牌移动到空格才是合法的移动)将初始排列转换目标排列（如图2）。

1	3	4	15
2		5	12
7	6	11	14
8	9	10	13

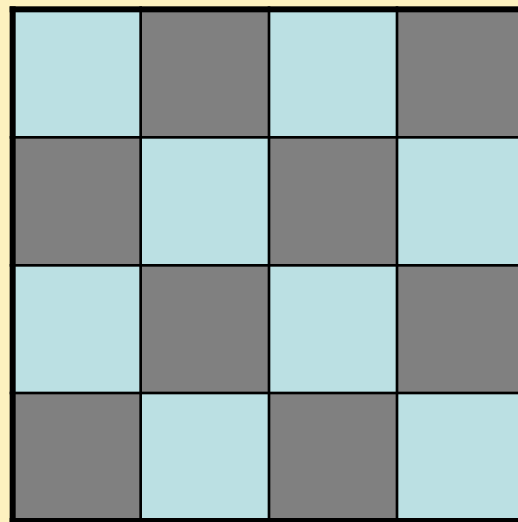
图1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

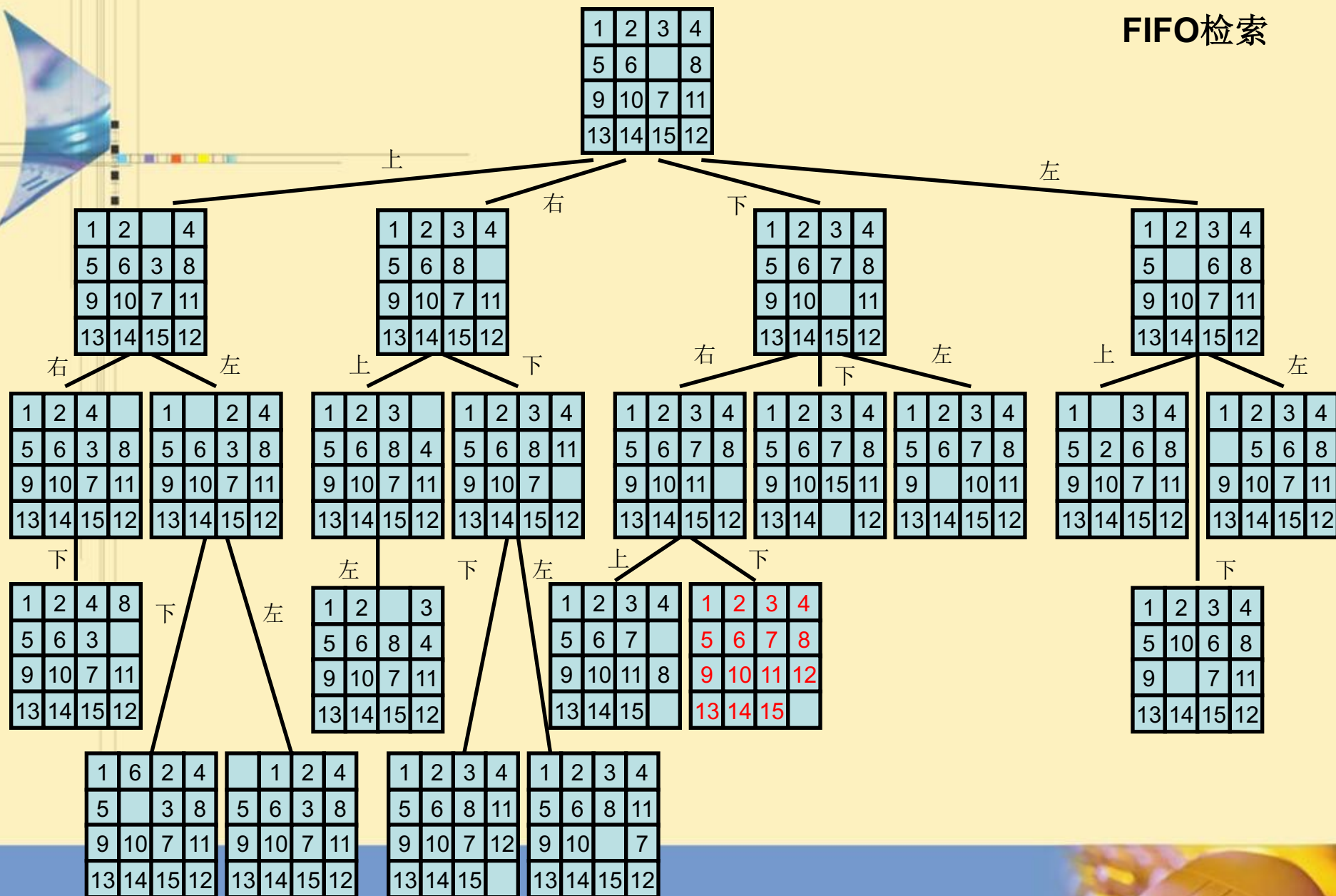
图2

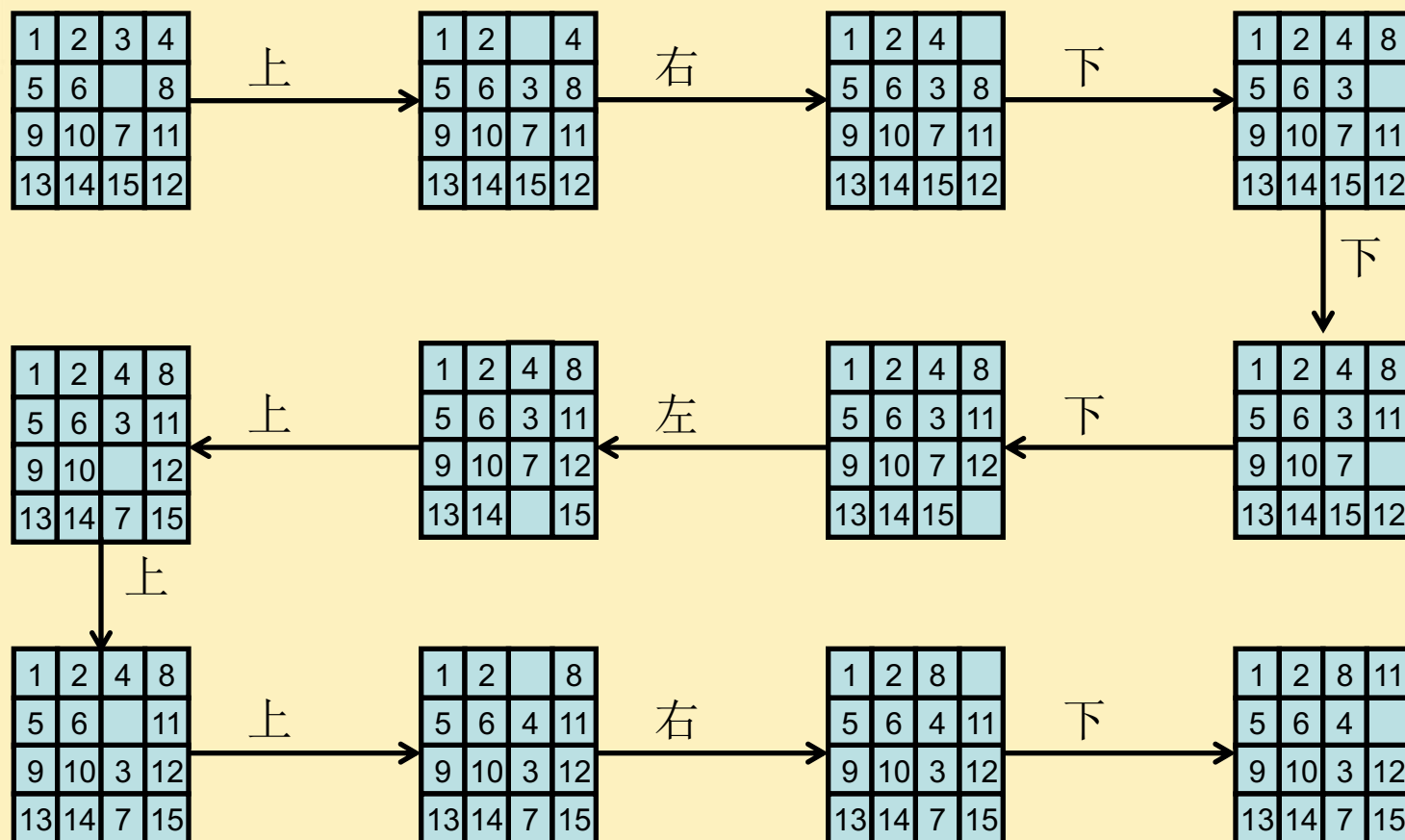
举例：15-谜问题

- 目标状态是否可由初始状态到达(即目标状态是否在初始状态的状态空间中)的判定:
 - 给棋盘的方格编上1~16的编号,位置*i*是目标排列中放*i*号牌的方格位置,位置16是空格的位置;
 - 设POSITION(*i*)表示编号为*i*的牌在初始状态下的位置号, $1 \leq i < 16$, POSITION(16)表示空格的位置;
 - 设LESS(*i*)是使牌*j*小于牌*i*且 POSITION(*j*) > POSITION(*i*)的数目;
 - 在初始状态下,如果空格在右图中阴影位置中的某一格处,则令X=1,否则令X=0。



定理：当且仅当 $LESS(1)+LESS(2)+\dots+LESS(16)+X$ 是偶数时，目标状态可由初始状态到达。





克服FIFO/LIFO检索缺陷的方法

- 设计一个“有智力的”排序函数 $c(.)$ ，对活结点进行排序，使最有可能产生答案结点的活结点优先成为E-结点；
- 将活结点组织成优先级队列，选取优先级最高的活结点作为下一个E-结点，从而可以加快到达一个答案结点的检索速度。
- 如果排序函数 $c(.)$ 定义为到答案所需要的耗费，则对应的检索方法为最小成本 (least cost) 检索（成本越小优先级越高），简称LC-检索。

结点的成本函数

- 定义一个结点的成本函数，对任一结点 X ，计算其找到答案结点要付出的代价，计算代价的度量标准：
 - (1) 在生成一个答案结点之前，子树 X 需要生成的结点数；或者
 - (2) 在子树 X 中离 X 最近的那个答案结点到 X 的路径长度。
- 结点的真实成本函数 $c(.)$ 定义如下：
 - 如果 X 是答案结点，则 $c(X)$ 是由状态空间树的根结点到 X 的成本(即所用的代价，它可以是级数，计算复杂度等)；
 - 如果 X 不是答案结点且子树 X 不包含任何答案结点，则 $c(X)=\infty$ ；否则 $c(X)$ 等于子树 X 中具有最小成本的答案结点的成本。

成本估计函数

结点的真实成本计算面临的问题：

- 要得到结点成本函数 $c(\cdot)$ 所用的计算工作量与解原问题具有相同的复杂度，这是因为计算一个结点的代价通常要检索包含一个答案结点的子树 X 才能确定，而这正是解决此问题所要做的工作。因此，要得到精确的成本函数一般是不现实的。

解决途径：定义成本估计函数： $\hat{c}(X)$ 。该函数强使算法优先选择更靠近答案结点但又离根较近的结点。

成本估计函数

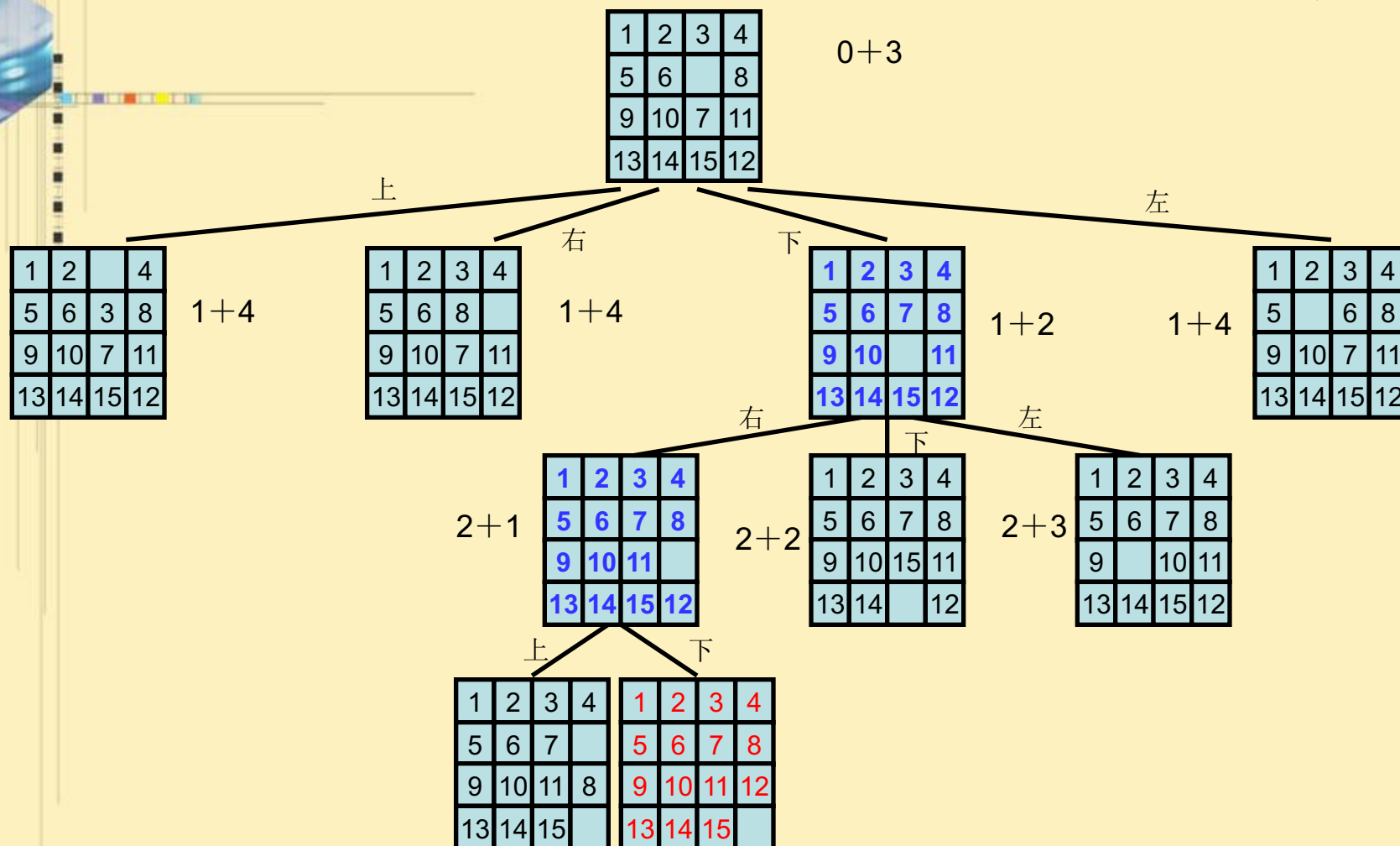
- 基于A*算法的启示，单纯只考虑从结点X到答案结点的估计成本会导致算法偏向于做纵深检查。
- 因此，为了得到具有最小成本的答案，对结点X的成本估计一方面要对从结点X到一个答案所需要的成本进行估计，另一方面，还要考虑从根结点到X的已经花费的成本。
- 结点的成本估计函数： $\hat{c}(X)=f(h(X))+\hat{g}(X)$ ，如果X是一个答案结点或是一个叶子结点，则有 $c(X)=\hat{c}(X)$
 - $\hat{g}(X)$ 是由X到达一个答案结点所需做的附加工作的估计；
 - $h(X)$ 是由根结点到结点X的成本。
 - $f(\cdot)$ 是一个非降函数。 f 函数强使算法优先选择更靠近答案结点但又离根较近的结点。它也能调整 $h(X)$ 和 $\hat{g}(X)$ 在 $\hat{c}(X)$ 中的影响比例。
- LC-检索：总是选取 $\hat{c}(\cdot)$ 值最小的活结点作为下一个E-结点的检索策略称为最小成本检索，简称为LC-检索。

LC-检索的特殊情况

- 若 $f(h(x)) = \text{结点} X \text{的级数}$ ，且 $\hat{g}(X) = 0$ 。如果结点的 $\hat{c}$ 值相同，选取最先生成的活结点。**LC-检索就是BFS检索**（队列实现的宽度优先搜索）。
- $f(h(x)) = 0$ ，且每当 Y 是 X 的一个子结点时，总有 $\hat{g}(X) \geq \hat{g}(Y)$ 。如果结点的 $\hat{c}$ 值相同，选取最晚生成的活结点，**LC-检索就是D-检索**（堆栈实现活结点表）

例：15-谜问题的LC-检索

- 成本函数 $c(\cdot)$: 从根出发到最近目标结点路径上的每个结点赋予这条路径的长度作为它们的成本.
- 估计成本函数 $\hat{c}(\cdot)$: $\hat{c}(X)=h(X)+\hat{g}(X)$,
 - $h(X)$ 是由根到结点 X 的路径的长度,;
 - $\hat{g}(X)$ 是对以 X 为根的子树中由 X 到目标状态的一条路径长度的估计。它至少应是能把状态 X 转换成目标状态所需的最小移动数。
$$\hat{g}(X)=\text{不在其目标位置的非空白牌数目}$$
- 可以看出 $\hat{c}(X)$ 是 $c(X)$ 的下界。



算法 LC-SEARCH

Input: 状态空间树T, T中结点的估计成本函数 $\hat{C}$ 。

Output: 问题的解 (从根到答案结点的路径)

1. if T 是答案节点 then 输出T and return; end if
2. $E \leftarrow T$ //E-结点
3. 将活结点表初始为空
4. repeat
5. for E 的每个儿子X
6. if X 是答案结点 then
7. 输出从X到T的路径, return
8. end if
9. call ADD(X) //将新的活结点X加到活结点表中
10. PARENT(X) $\leftarrow$ E //E结点标记为X的父结点
11. end for
12. if 活结点表为空 then
13. 输出“无解”, return
14. end if
15. call LEAST(E) //找一个具有最小 $\hat{C}$ 值的活结点放到E中, 从活结点表中删除
16. end repeat

LC-SEARCH的特性分析

1. 如果有两个结点 X, Y , $c(X) > c(Y)$, 但 $\hat{c}(X) < \hat{c}(Y)$, LC-SEARCH 不能达到具有最小成本的答案结点;
 2. 如果使每一对 $c(X) < c(Y)$ 的结点 X, Y , 都有 $\hat{c}(X) < \hat{c}(Y)$, 则在一棵至少有一个答案结点的有限状态树中, 算法总会找到一个最小成本的答案结点。但在实际应用中, 定义满足该要求且易于计算的 $\hat{c}(\cdot)$ 通常很难。
- 在实际应用中, 一般只能找到易于计算且有如下性质的 $\hat{c}(\cdot)$:
 - (1) 对每个结点 X , 总有 $\hat{c}(X) \leq c(X)$
 - (2) 对于答案结点 X , 有 $\hat{c}(X) = c(X)$.
 - 这时改进算法LC-SEARCH, 可找到最小成本的答案结点。

算法 LC1-SEARCH

Input: 状态空间树T, T中结点的估计成本函数 $\hat{C}$ 。

Output: 问题的解 (从根到最小成本答案结点的路径)

1. $E \leftarrow T$ //E-结点
2. 将活结点表初始为空
3. repeat
4. if E 是答案节点 then 输出从E到T的路径 and return; end if
5. for E 的每个儿子X
6. call ADD(X) //将新的活结点X加到活结点表中
7. PARENT(X) $\leftarrow$ E //E结点标记为X的父结点
8. end for
9. if 活结点表为空 then
10. 输出“无解”, return
11. end if
12. call LEAST(E) //找一个具有最小 $\hat{C}$ 值的活结点放到E中, 从活结点表中删除
13. end repeat



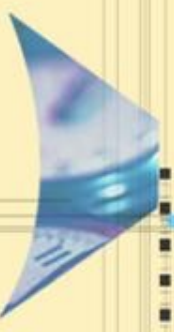
主要内容

9.1 引言

9.2 检索策略

9.3 分支限界算法设计思想

9.4 分支限界法的典型应用



主要内容

9.3 分支限界算法设计思想

9.3.1 分支限界法的基本思想

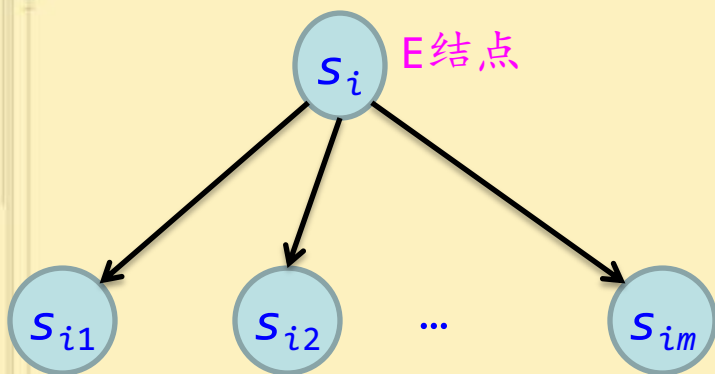
9.3.2 找最小成本答案结点的分支限界法

9.3.3 求最优解的分支限界算法



9.3.1 分支限界法的基本思想

所谓“分支”就是采用广度优先的策略，依次生成当前E结点的所有分支，也就是所有子结点。将生成的子结点放入活结点表中。



产生所有的子结点

选择哪一个活结点？

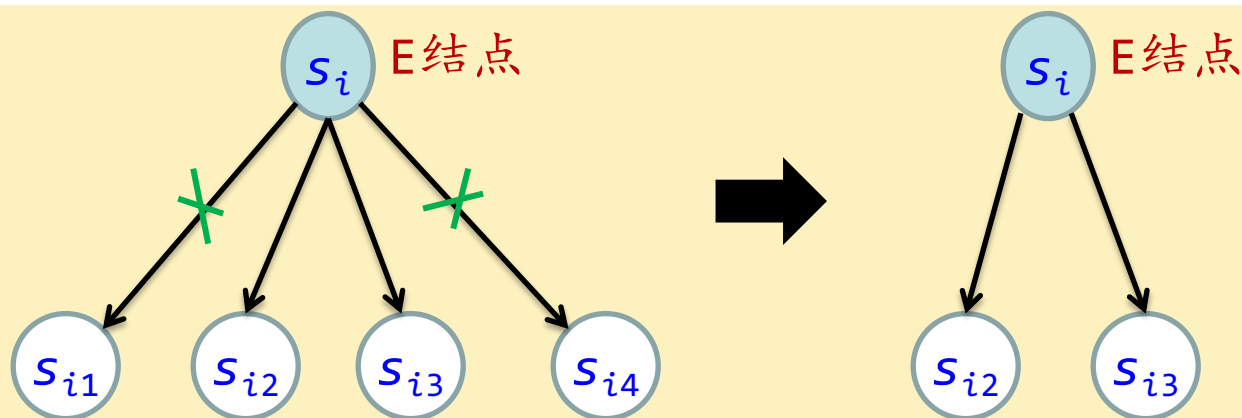
- FIFO检索
- 最小耗费或者最大效益检索
(设计代价或者效益函数)

所谓“限界”，就是设计一个限界函数，删除不能到达解或者无法到达最优解的结点。从而加速搜索进程。

(1) 设计合适的限界函数

- 在搜索解空间树时，每个活结点可能有很多子结点，其中有些子结点搜索下去是不可能产生问题解或最优解的。因此，可以设计好的限界函数在扩展时删除这些不必要的子结点，从而提高搜索效率。

例如，假设结点 s_i 有4个子结点，而满足限界函数的子结点只有2个，可以删除另外2个不满足限界函数的子结点，使得从 s_i 出发的搜索效率提高一倍。

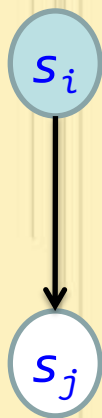


(1) 设计合适的限界函数

限界函数设计难以找出通用的方法，需根据具体问题来分析。
一般地，先要确定问题解的特性。

对于优化问题：

- **目标函数是求最大值：**则设计上界限界函数ub（根结点的ub值通常大于或等于最优解的ub值），若 s_i 是 s_j 的父结点，应满足 $ub(s_i) \geq ub(s_j)$ ，当找到一个可行解后，将所有小于当前最好解的值的结点剪枝。
- **目标函数是求最小值：**则设计下界限界函数lb（根结点的lb值一定要小于或等于最优解的lb值），若 s_i 是 s_j 的父结点，应满足 $lb(s_i) \leq lb(s_j)$ ，当找到一个可行解后，将所有大于当前最好解的值的结点剪枝。

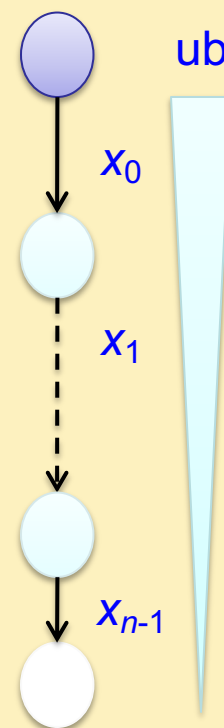


假设解向量 $x = (x_0, x_1, \dots, x_{n-1})$ ，如果目标函数是求**最大值**

设计**上界限函数** $ub()$ ，若从 x_i 的分支向下搜索所得到的部分解是 $(x_0, x_1, \dots, x_i, \dots, x_k)$ ，则应该满足：

$$ub(x_i) \geq ub(x_{i+1}) \geq \dots \geq ub(x_k)$$

所以根结点的**ub**值应该大于或等于最优解的**ub**值。

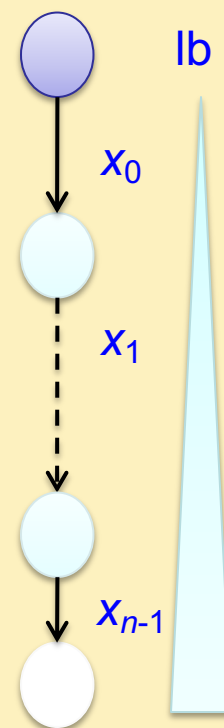


假设解向量 $x = (x_0, x_1, \dots, x_{n-1})$ ，如果目标函数是求**最小值**

设计下界限函数**lb()**，若从 x_i 的分支向下搜索所得到的部分解是 $(x_0, x_1, \dots, x_i, \dots, x_k)$ ，则应该满足：

$$lb(x_i) \leq lb(x_{i+1}) \leq \dots \leq lb(x_k)$$

所以根结点的**bb**值应该小于或等于最优解的**lb**值。



(2) 活结点的组织

根据选择下一个扩展结点的方式来组织活结点表，不同的活结点表对应不同的分支搜索方式，常见有两种分支限界法（LIFO 不常见）：

(1) 队列式(FIFO)分支限界法

按照队列先进先出（FIFO）原则选取下一个活节点为E-节点。

(2) 优先队列式分支限界法

按照优先队列中规定的优先级选取优先级最高的活节点成为E-节点。

队列式分支限界法

队列式分支限界法将活结点表组织成一个队列，并按照队列先进先出（FIFO）原则选取下一个结点为扩展结点。步骤如下：

- ① 将根结点加入活结点队列。
- ② 从活结点队中取出队头结点，作为当前扩展结点。
- ③ 对当前扩展结点，先从左到右地产生它的所有子结点，用约束条件检查，把所有满足约束条件的子结点加入活结点队列。
- ④ 重复步骤②和③，直到找到一个解或活结点队列为空为止。

优先队列式分支限界法

优先队列式分支限界法的主要特点是将活结点表组组成一个优先队列，并选取优先级最高的活结点成为当前扩展结点。步骤如下：

- ① 计算起始结点（根结点）的优先级并加入优先队列（与特定问题相关的信息的函数值决定优先级）。
- ② 从优先队列中取出优先级最高的结点作为当前扩展结点，使搜索朝着解空间树上可能有解或最优解的分支推进，以便尽快地找出一个解或最优解。
- ③ 对当前扩展结点，先从左到右地产生它的所有子结点，然后用约束条件检查，对所有满足约束条件的子结点计算优先级并加入优先队列。
- ④ 重复步骤②和③，直到找到一个解或优先队列为空为止。

9.3.2找最小成本答案结点的分支限界算法

- 检索状态空间树的各种分支限界算法都是在生成当前E-结点的所有儿子结点后，再从所有活结点中检索一个变成E-结点。
- 假定每个答案结点X有一个与其联系的成本 $c(X)$ ，并且假定会找到最小成本的答案结点。
- 那么，可使用一个使得 $\hat{c}(X) \leq c(X)$ 的成本估计函数 $\hat{c}(\cdot)$ 来给出由任一结点X得出的解的下界；
- 采用下界函数，可使算法具有一定的智能，从而减少盲目性；
- 另外，如前分析，为了利用分支限界法，估计函数需要满足性质：如果 X_j 是 X_i 扩展出的结点，则 $\hat{c}(X_i) \leq \hat{c}(X_j)$ 。因此如果扩展出来的结点的估计值大于先前得到的解的值，则可以去掉。
- 同时，可以设置最小成本的上界，使算法进一步加速。

9.3.2找最小成本答案结点的分支限界算法

- 定义 U 是最小成本解的成本上界, 则:
 - (1) 具有 $\hat{c}(X) > U$ 的活结点 X 可以被杀死。这是因为由 X 可以到达的答案结点 X 有 $c(X) \geq \hat{c}(X) > U$;
 - (2) 如果已经到达一个具有成本 U 的答案结点, 则 $\hat{c}(X) \geq U$ 的所有活结点都可以被杀死;
- U 的初始值可以通过某种启发式方法得到, 也可以设为 ∞ ;
- 显然, 只要 U 的初始值不小于最小成本答案结点的成本, 上述杀死活结点的规则不会去杀死可以达到最小成本答案结点的活结点。
- 每当找到一个新的解, 就可以修改 U 的值。

算法 FIFOBB

Input: 状态空间树T, T中结点的估计成本函数 $\hat{c}$, 可行解的成本函数cost, $\varepsilon > 0$, 上界函数u。

Output: 最小成本的解 (从根到最小成本答案结点的路径)

// 为找出最小成本的结点检索T。假设T至少包含一个可行解结点, 且有 $\hat{c}(X) \leq c(X) \leq u(X)$

// ε 为足够小的值, 使得对任两个可行结点X和Y, 如果 $u(X) < u(Y)$, 则 $u(X) < u(X) + \varepsilon < u(Y)$

1. $E \leftarrow T$; $PARENT(E) \leftarrow 0$

2. if T是可行解结点 then

3. $U \leftarrow \min(\text{cost}(T), u(T) + \varepsilon)$; $\text{ans} \leftarrow T$ //U代表目前最好解的值

4. else

5. $U \leftarrow u(T) + \varepsilon$; $\text{ans} \leftarrow 0$

6. end if

7. 将队列表初始化为空

6. while true

7. for E 的每一个儿子X

8. if $\hat{c}(X) < U$ then

9. $\text{ADDQ}(X)$; $PARENT(X) \leftarrow E$

10. if X 是可行解结点 and $\text{cost}(X) < U$ then $U \leftarrow \min(\text{cost}(X), u(X) + \varepsilon)$; $\text{ans} \leftarrow X$

11. else if $u(X) + \varepsilon < U$ then $U \leftarrow u(X) + \varepsilon$

12. end if

13. end if

14. end for

15. while true //得到下一个E结点

16. if 活结点表为空 then

17. 输出从ans到T的路径; 输出最小成本U; return

18. end if

19. $\text{DELETEQ}(X)$ //队列中删除一个结点X

20. if $\hat{c}(X) < U$ then $E \leftarrow X$; exit; end if //杀死 $\hat{c}(X) \geq U$ 的结点

21. end while

22. end while

算法 LCBB

Input: 状态空间树T, T中结点的估计成本函数 $\hat{c}$, 可行解的成本函数cost, $\varepsilon > 0$, 上界函数u。

Output: 最小成本的解 (从根到最小成本答案结点的路径)

// 为找出最小成本的结点检索T。假设T至少包含一个可行解结点, 且有 $\hat{c}(X) \leq c(X) \leq u(X)$

// ε 为足够小的值, 使得对任两个可行结点X和Y, 如果 $u(X) < u(Y)$, 则 $u(X) < u(X) + \varepsilon < u(Y)$

1. $E \leftarrow T$; $PARENT(E) \leftarrow 0$

2. if T是可行解结点 then

3. $U \leftarrow \min(\text{cost}(T), u(T) + \varepsilon)$; $\text{ans} \leftarrow T$

4. else

5. $U \leftarrow u(T) + \varepsilon$; $\text{ans} \leftarrow 0$

6. end if

7. 将活结点表初始化为空

6. while true

7. for E 的每一个儿子X

8. if $\hat{c}(X) < U$ then

9. ADD(X); $PARENT(X) \leftarrow E$

10. if X 是可行解结点 and $\text{cost}(X) < U$ then $U \leftarrow \min(\text{cost}(X), u(X) + \varepsilon)$; $\text{ans} \leftarrow X$

11. else if $u(X) + \varepsilon < U$ then $U \leftarrow u(X) + \varepsilon$

12. end if

13. end if

14. end for

15. if 活结点表为空 or 下一个E结点E有 $\hat{c}(E) \geq U$ then

16. 输出从ans到T的路径; 输出最小成本U; return

17. end if

18. LEAST(E) //选最小活结点作为E结点

19. end while

9.3.3 求最优解的分支限界算法

最小化问题：基于求最小成本的分支限界算法思路

- 将对最小成本答案结点的检索转换为对最优解的检索。因此，需要将成本函数定义为满足最优解的答案结点 X 的成本值 $c(X)$ 是所有结点成本的最小值。
- 最简单的方法就是直接把目标函数作为成本函数。
- 因此，代表可行解的结点 X 的成本值 $c(X)$ 就是那个可行解的目标函数值；代表不可行解的结点的 $c(X)=\infty$ ；代表部分可行解的结点的 $c(X)$ 是根为 X 的子树中最小成本结点的成本。由于计算 $c(X)$ 和求出最优解难度一样，因此，定义一个估值函数 $\hat{c}(\cdot)$ ，对所有结点 X ， $\hat{c}(X) \leq c(X)$ ，并且对于答案结点 $\hat{c}(X)=c(X)$ 。可见，在求最小化问题中， $\hat{c}(\cdot)$ 是对目标函数值进行估计，而不是对找到答案结点的计算量进行估计。

9.3.3 求最优解的分支限界算法

最小化问题：基于求最小成本的分支限界算法思路

- 定义一个目标函数的估值函数 $\hat{c}(\cdot)$ ，对所有结点 X ， $\hat{c}(X) \leq c(X)$ ，并且对于答案结点 $\hat{c}(X) = c(X)$ 。函数值是以该结点为根的树中的所有可行解的目标函数值的下界。
- 定义最优值的上界：初始值根据启发式方法确定，或者设为足够大。当搜索到一个可行解时，如果其目标函数值小于当前的上界，就更新为更小的上界。
- 剪枝策略：如果扩展出来的结点的估计值大于先前得到的最好解的值 $best$ ，则可以去掉。
- 最优值的上界更新：初始时通过启发式方法确定或者设定为足够大，如果目标函数值为负，则初始值可设为0。当搜索到一个答案时，如果其目标函数值小于当前的上界，就更新为更小的上界。

9.3.3 求最优解的分支限界算法

最大化问题，两种思路：

- (1) 转换成最小化问题：改变目标函数的符号等
- (2) 直接求最大化问题



最大化问题的分支限界算法

- 对于最大化问题，可以将求最小化问题的内容进行对偶转换
- 定义目标函数估计函数（代价函数）：函数值是以该结点为根的树中的所有可行解的目标函数值的上界。即： $\hat{c}(X) \geq c(X)$ ，并且对于答案结点 $\hat{c}(X) = c(X)$ 。显然，父结点的代价 $\geq$ 子结点的代价
- 定义最优值的下界：其值是当前已经得到的答案的目标函数值的最大值。
- 杀死结点策略：如果某结点不满足约束条件，或者代价函数值小于当前最优值的下界 L ，则杀死该结点。
- 最优值的下界更新：初始时通过启发式方法确定或者设定为足够小，如果目标函数值为正，则初始值可设为0。当搜索到一个答案时，如果其目标函数值大于当前的下界，就更新为更大的下界。

算法 UCBB

Input: 状态空间树 T , T 中结点的代价函数 $\hat{c}$, 可行解的函数 cost , $\epsilon > 0$, 下界函数 lb 。

Output: 函数值最大的解

// 为找出最小成本的结点检索 T 。假设 T 至少包含一个可行解结点, 且有 $\text{lb}(X) \leq c(X) \leq \hat{c}(X)$

// ϵ 为足够小的值, 使得对任两个可行结点 X 和 Y , 如果 $\text{lb}(X) < \text{lb}(Y)$, 则 $\text{lb}(X) - \epsilon < \text{lb}(X) < \text{lb}(Y)$

1. $E \leftarrow T$; $\text{PARENT}(E) \leftarrow 0$

2. **if** T 是可行解结点 **then**

3. $L \leftarrow \max(\text{cost}(T), \text{lb}(T) - \epsilon)$; $\text{ans} \leftarrow T$

4. **else**

5. $L \leftarrow \text{lb}(T) - \epsilon$; $\text{ans} \leftarrow 0$

6. **end if**

7. 将活结点表初始化为空

6. **while true**

7. **for** E 的每一个儿子 X

8. **if** $\hat{c}(X) > L$ **then**

9. $\text{ADD}(X)$; $\text{PARENT}(X) \leftarrow E$

10. **if** X 是可行解结点 **and** $\text{cost}(X) > L$ **then** $L \leftarrow \max(\text{cost}(X), \text{lb}(X) - \epsilon)$; $\text{ans} \leftarrow X$

11. **else if** $\text{lb}(X) - \epsilon > L$ **then** $L \leftarrow \text{lb}(x) - \epsilon$

12. **end if**

13. **end if**

14. **end for**

15. **if** 活结点表为空 **or** 下一个 E 结点 E 有 $\hat{c}(E) \leq L$ **then**

16. 输出从 ans 到 T 的路径; 输出最大值 U ; **return**

17. **end if**

18. $\text{LARGEST}(E)$ //选代价函数值最大的活结点作为 E 结点

19. **end while**